StoreIntel: AI-Powered Real-Time Store Intelligence & Customer Analytics System

Purplle Tech Challenge 2026 — Round 2 Technical Proposal

Submitted By: Shivam Giri
Role: Full-Stack & AI Engineer (Individual)
GitHub Link: https://github.com/shivamgiri068/StoreIntel
Submission Date: June 2, 2026

1. Executive Summary

Traditional brick-and-mortar retail spaces suffer from a data blindspot. While e-commerce platforms can track click-through rates, drop-offs, and session dwell times, physical retail managers rely on manual log count sheets and static POS records. **StoreIntel** bridges this gap. By utilizing existing CCTV security cameras, the system converts raw video streams into spatial customer journey metrics, locations of high dwell time, and operational anomaly alerts.

2. Technical Stack & Deployment Strategy

To ensure production-readiness under real-world constraints, StoreIntel utilizes a lightweight, containerized stack:

Layer	Technology Selection	Trade-Off Rationale
AI & Tracking	YOLOv8 Nano + ByteTrack	30 FPS execution on CPUs with Kalman filter tracking ID resilience.
API Framework	Python FastAPI (Port 8000)	High-throughput asynchronous request serving and Pydantic validation
Local Database	SQLite Database	Self-contained SQL engine, avoiding overhead of external databases.
Visualization	Streamlit App (Port 8501)	Rapid operational dashboards with native Plotly visualizers.
Orchestration	Docker Compose	Multi-container isolation ensuring simple deployment via one command

3. Edge Detection & Multi-Object Tracking

The computer vision pipeline processes standard frame inputs by filtering for the **Person class** (ID 0) and tracking coordinates through spatial mapping.

3.1 YOLOv8 Person Filter

We filter the standard COCO detections exclusively for Class ID 0. Detections are run at a confidence threshold of 0.30, allowing detection of occluded, side-view, or moving shoppers while discarding noise.

3.2 ByteTrack Kalman Associations

Rather than computing heavy deep-learning embeddings for Re-Identification, ByteTrack associates bounding boxes via motion predictions (Kalman Filters) combined with spatial overlaps (IoU). This keeps frame processing speeds above 30 FPS on edge CPUs.

3.3 Spatial Bottom-Middle Footprint

To map a customer's location to the store layout, we calculate the bottom-middle coordinate of their bounding box: **$(t_x, t_y) = (x_{min} + width/2, y_{max})$**. This maps to the shopper's feet contact point on the floor, rather than their geometric centroid which changes with head movements or posture.

4. Polygon Mapping & State Machine Logic

Using **Shapely** geometry checks, the system maps coordinate footprints to designated Store Zones (e.g., Entrance, Skincare Shelf, Makeup Shelf, Checkout Area).

4.1 Geometric Containment

A zone z is defined by a coordinate polygon P_z. A shopper centroid (t_x, t_y) is registered inside the zone if the point intersects the polygon check: **Point(t_x, t_y).within(Polygon(P_z))**.

4.2 State Machine Transition Rules

The system tracks the state of each active Shopper ID:
1. **First Seen**: Emit a **STORE_ENTRY** event to denote store entry.
2. **Zone Entry**: If the centroid enters polygon P_z, emit **ZONE_ENTER**.
3. **Zone Exit**: If the centroid leaves polygon P_z, emit **ZONE_EXIT** and calculate **DWELL_TIME**.
4. **Track Loss**: If a track is lost for more than 30 consecutive frames, emit a final **STORE_EXIT**.

4.3 Telemetry Event Schema Example

```
{ "eventId": "e932b130-1c39-44d4-9d51-4099496a798f", "eventType": "ZONE_ENTER",
"timestamp": "2026-06-01T23:15:00.000", "customerId":
"81a1795c-9c59-450f-90db-33cb397bdf01", "zoneName": "Skincare Shelf", "x": 310.5, "y":
320.0 }
```

5. REST API Documentation & Database

API endpoints serve analytics data to the operational dashboard in real-time, pulling directly from index-optimized SQLite tables.

5.1 Ingest & Analytics Endpoints

- **POST /events/ingest**: Receives telemetry payloads from the computer vision thread and stores them.
- **GET /metrics**: Aggregates total footfall, current store occupancy, and popular shelves.
- **GET /funnel**: Builds conversion metrics (Entrance ➔ Shelf ➔ Checkout).
- **GET /heatmap**: Returns coordinates mapping density tracking values.
- **GET /anomalies**: Serves security loitering or queue backup alerts.

5.2 SQLite Database Tables Schema

```
CREATE TABLE events (
    id TEXT PRIMARY KEY,
    event_type TEXT,
    timestamp TEXT,
    customer_id TEXT,
    zone_name TEXT,
    x REAL, y REAL,
    metadata TEXT
);
CREATE TABLE anomalies (
    id TEXT PRIMARY KEY,
    anomaly_type TEXT,
    severity TEXT,
    timestamp TEXT,
    description TEXT,
    resolved INTEGER DEFAULT 0
);
```

6. Operational Anomaly Calculations

The system evaluates operational anomalies asynchronously to issue real-time alerts:

6.1 Queue Congestion (Crowd Surge)

Rule: Evaluates unique active Customer IDs inside the checkout area polygon over the last 2 minutes. If counts exceed 3 customers, a high-severity anomaly alert is written to the database.

6.2 Customer Loitering

Rule: Triggers if a customer's cumulative elapsed dwell time inside a shelf zone exceeds 15 seconds (representing a real-world threshold of 10 minutes), flagging potential loitering or staff-assistance requirements.

6.3 CCTV Outage/Failure

Rule: Triggers if the duration elapsed since the last received event (delta T) exceeds 180 seconds, raising a system connection failure warning in the dashboard diagnostics panel.

7. Architectural Trade-off Summary

1. **Decoupled Architecture**: Vision execution runs inside an isolated background worker thread, ensuring the API serving thread remains responsive and latency stays below 5ms.
2. **SQLite Selection**: Zero-configuration SQLite local database eliminates multi-container orchestration bottlenecks, keeping memory consumption below 500MB.
3. **Simulation Mode Fallback**: If CCTV clips are not loaded in the videos directory, the system starts a simulation engine to feed mock telemetry so that the client dashboard runs immediately without crashes.